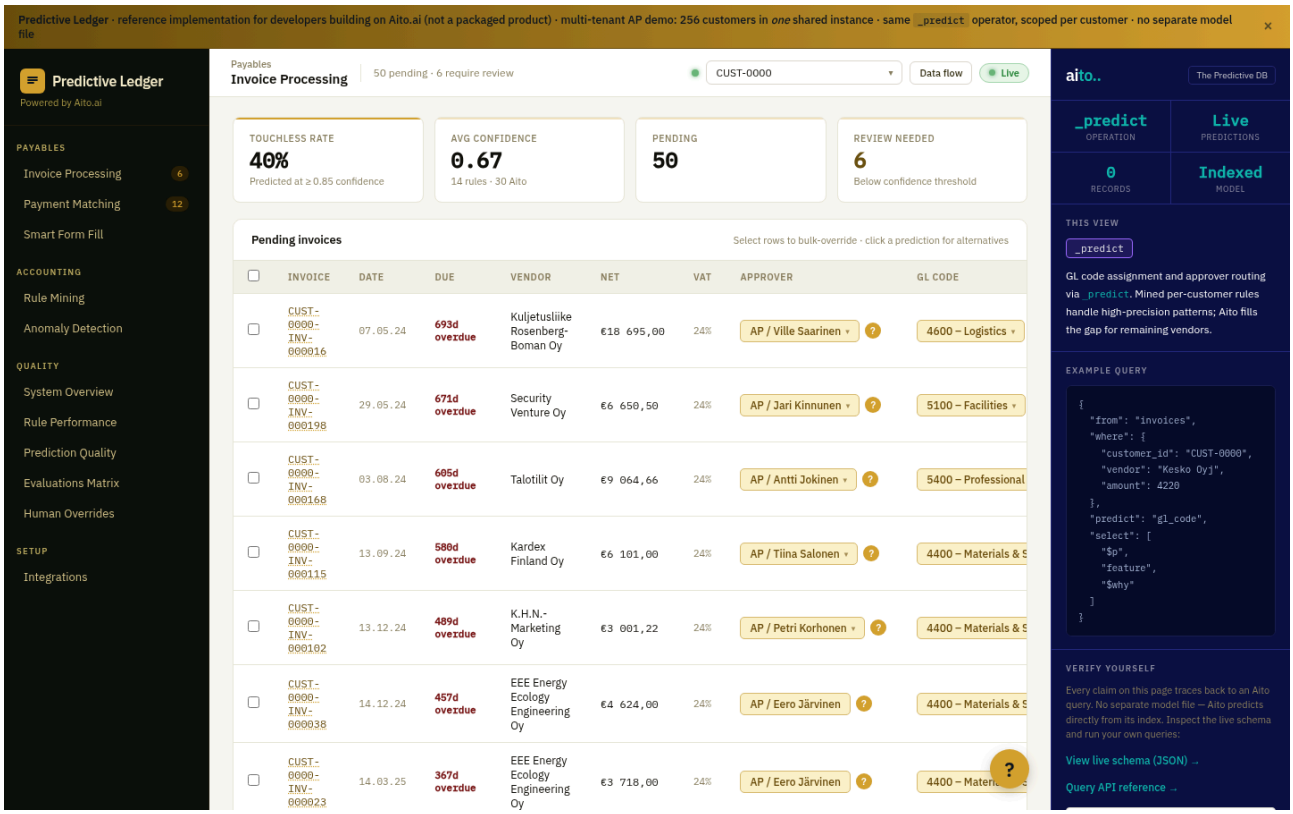
Aito.ai · Reference implementation

Predictive Ledger

Multi-tenant accounts-payable automation on a single shared Aito instance.
255 customer companies. 128 000 invoices. One `customer_id` filter.
Same `_predict` operator at every scale — no per-tenant model, no retraining.



The Challenge

SaaS accounting products live in the gap between two hard problems: rules-based automation tops out around 70%, and the long tail that remains is too sparse and too contextual for hand-written rules. A typical AP team books the easy 7 invoices in 10 from a ruleset and reviews the other 3 by hand.

❌ Per-tenant ML doesn't scale

One model per customer means one training pipeline per customer. Operationally untenable for a SaaS with hundreds of tenants — and the smallest customers have too little data to train against.

🪤 Rules can't cover the long tail

Hand-coded routing covers high-frequency vendors. The remaining 30% — new vendors, edge categories, amount escalations — is where the AP team's time goes.

📋 Black-box models fail audit

SOX and procurement reviews require explainable routing. Most ML systems produce a probability and a number; auditors want a reason, in plain language, per decision.

The Solution

Aito's predictive database treats every prediction as a conditional probability over the indexed table. Add `customer_id` to the `where` clause and the same operator works per-tenant. No per-customer model file, no nightly training, no deployment pipeline beyond data ingest.

🏢 Single shared instance

255 customers in one `invoices` table. Multi-tenancy via where-clause. New customers added like new rows.

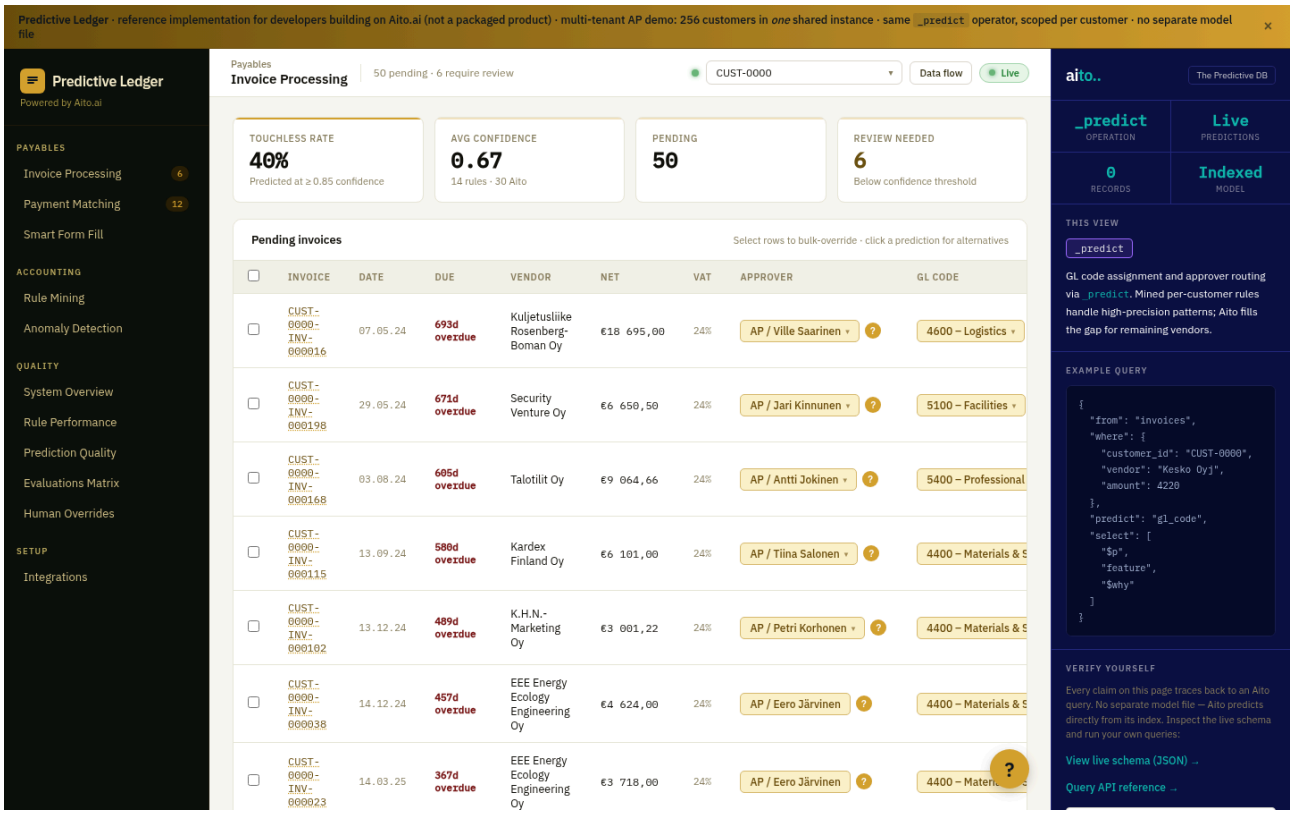
📊 Honest uncertainty

Cold-start customer with 125 invoices? Confidence drops to 0.2-0.4 — not faked. Top-tier customer with 16K invoices? 98% accuracy.

🔍 Audit-grade explanations

Every prediction includes the base rate × pattern lifts × final probability. Token highlights show which invoice description words drove the GL match.

Multi-tenancy in one query



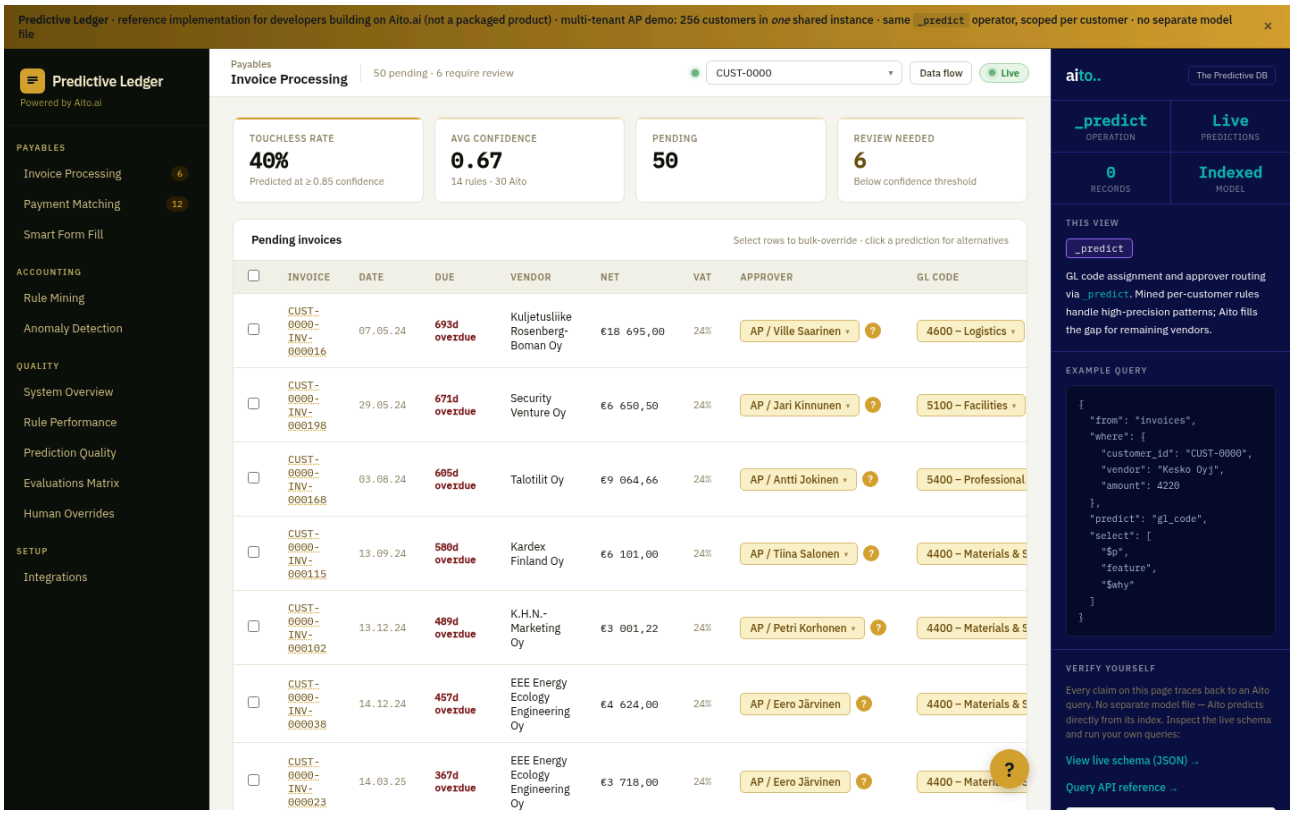
The topbar customer switcher is the demo's headline gesture. Same vendor (Kardex Finland Oy) under three different customers' contexts returns three different GL predictions:

CUST-0000 · enterprise · 16K invoices ⟶ GL 4400 (97%)
CUST-0003 · large · 4K invoices ⟶ GL 6200 (82%)
CUST-0100 · small · 250 invoices ⟶ GL 6200 (67%)

`where: {customer_id, vendor}` is the entire isolation mechanism. Adding the customer filter reduces the row set the conditional probability is computed over — Aito's index handles it as just another constraint.

No per-tenant model file, no training pipeline, no schema split, no isolation layer beyond the where clause.

Predictions with explanations

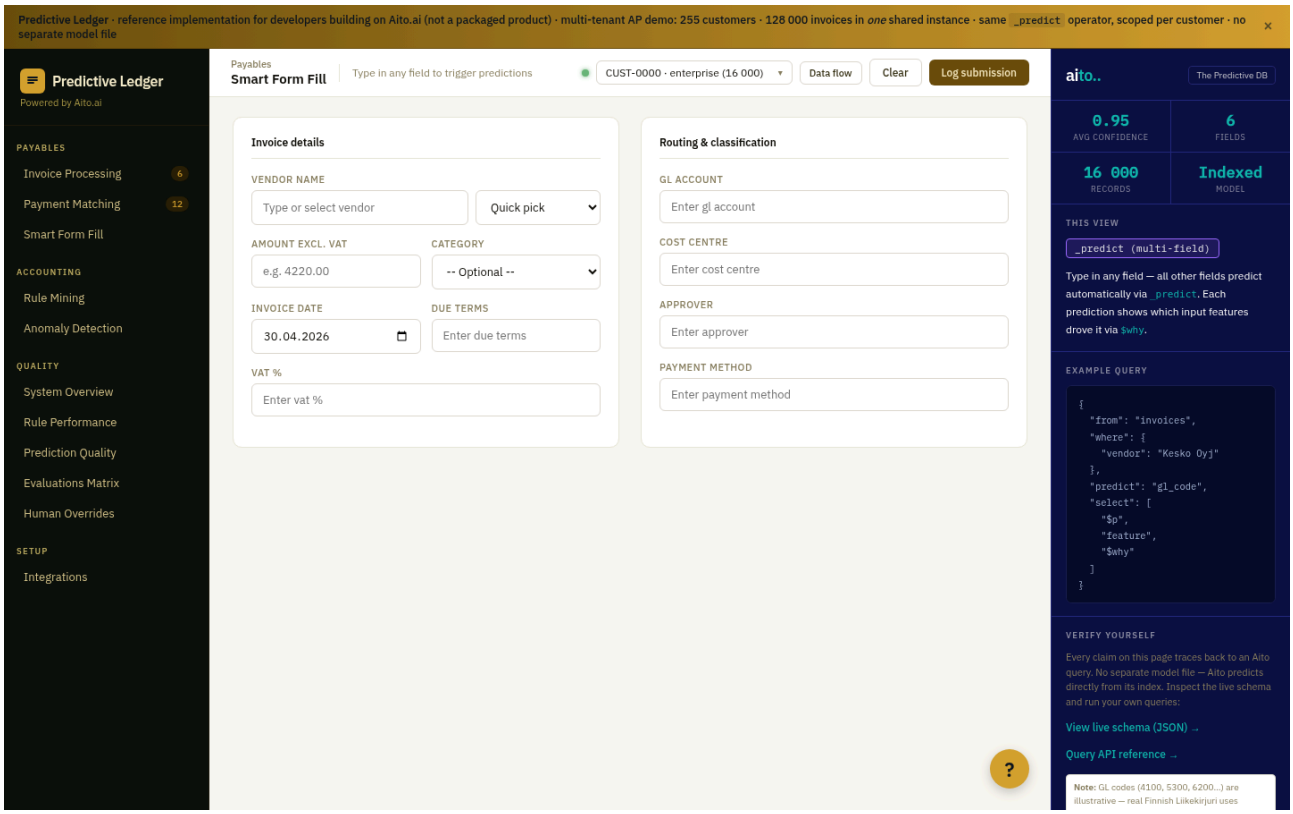


Every GL code and approver prediction in the invoice list opens a detail dock with the full `$why` factor breakdown:

Base probability — 17% historical rate for GL 4400
Pattern match — ×5.7 when vendor = "Kardex Finland Oy" **and** category = "supplies"
Pattern match — ×2.0 when description contains "Office", "supplies", "bulk"
Result — 17% × 5.7 × 2.0 = 97% confidence

Token-level highlighting shows the exact words that drove a Text-field match. Hovering a factor highlights the source field on the left side of the dock so operators can verify the explanation against the input.

Smart Form Fill

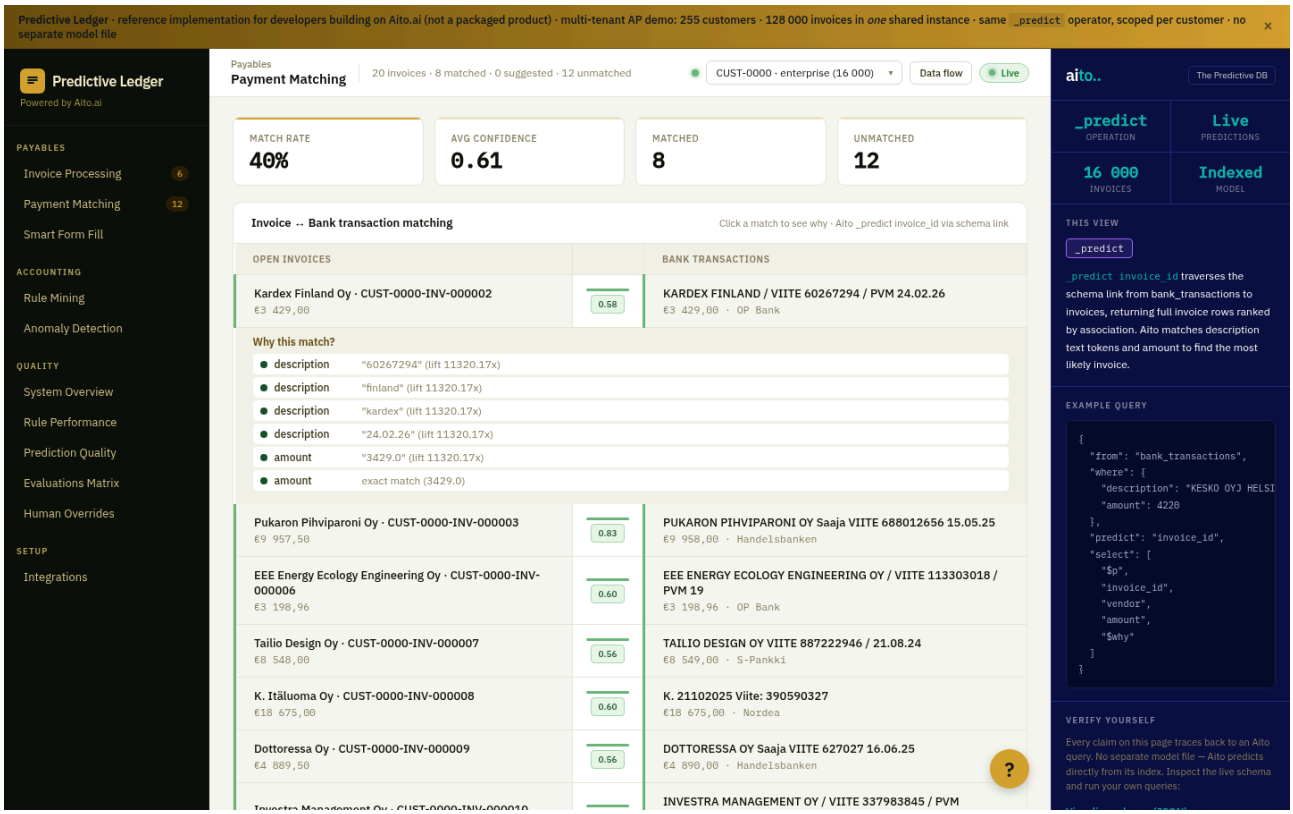


Type any field, the rest predict. Six fields (GL, approver, cost centre, payment method, due terms, VAT %) populate from a single chosen vendor in 150 ms. Each prediction shows top-3 alternatives plus `$why` factors.

Demo path: select Kardex Finland Oy ⟶ fields populate at 90%+ confidence. Bump the amount to €50 000 ⟶ approver re-predicts to a senior signer (amount-escalation pattern picked up from history).

What it answers: "How much does Aito reduce manual data entry?" On the demo dataset, six of seven invoice fields auto-fill at high confidence after vendor selection.

Payment matching

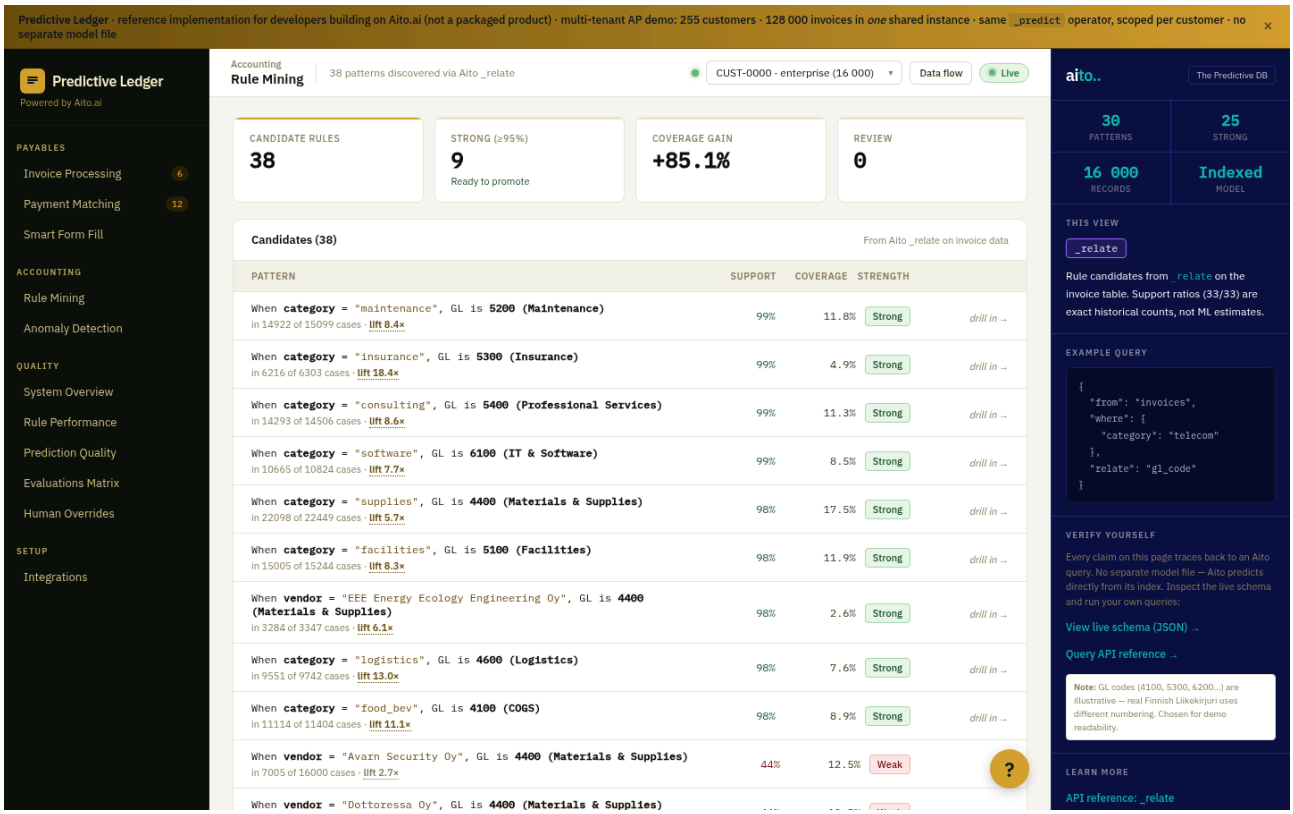


Bank descriptions are messy text. `_predict invoice_id` over the `bank_transactions` table handles the matching with no hand-coded regex — Aito's analyzer tokenizes the description and matches via shared tokens with the linked `invoices.invoice_id`.

```
from: "bank_transactions"
where: { customer_id: "CUST-0000",
         description: "OP /VENDOR/ KARDEX FINLAND...",
         amount: 2158 }
predict: "invoice_id"
select: ["$p", "invoice_id", "vendor", "amount"]
```

The schema link makes `invoice_id` a foreign key — Aito returns the linked invoice's `vendor` and `amount` in the same response. No manual join, no second query.

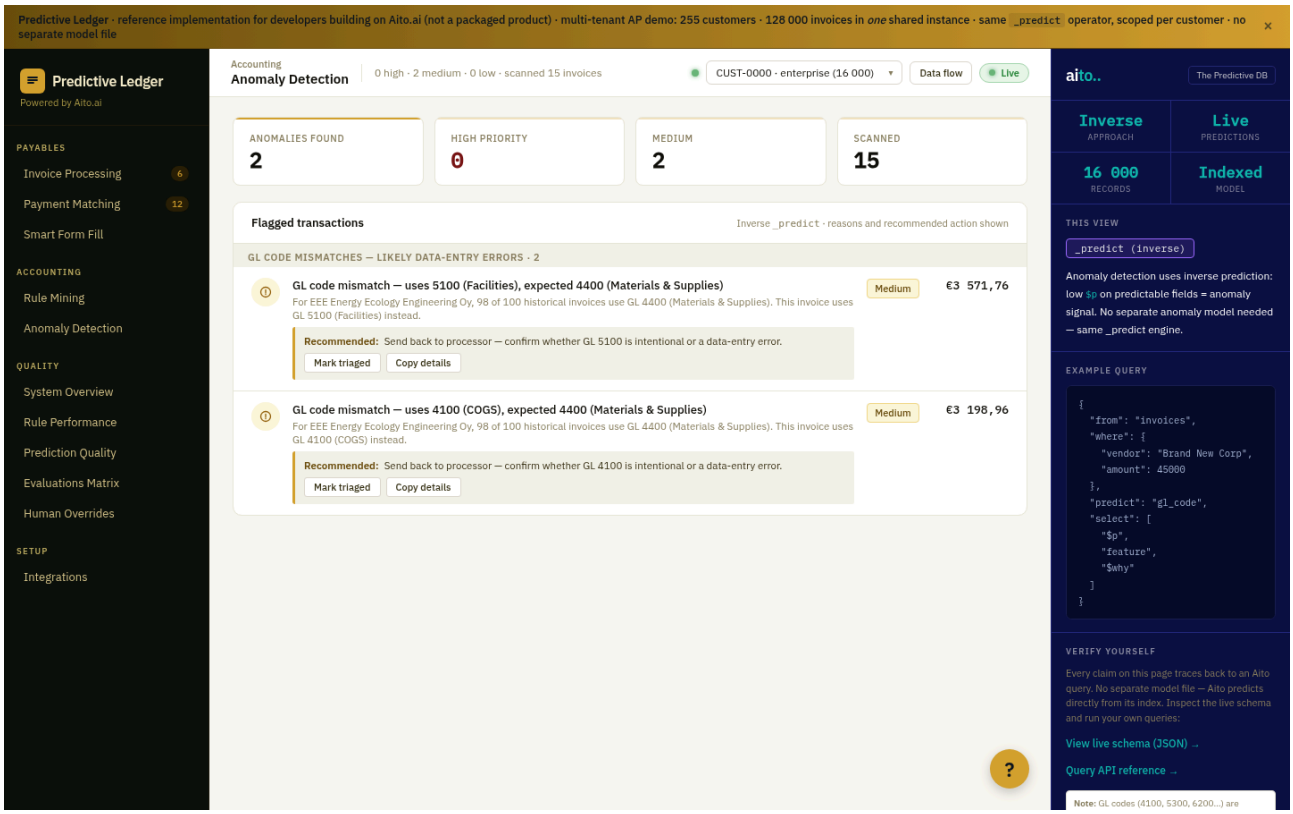
Rule mining



`_relate` finds deterministic patterns ("when vendor=X, GL is Y in 4123/4156 cases, lift 38×"). Each candidate row expands inline into a chained `_relate` for compound sub-patterns: `category=telecom & gl_code=6200 → approver=Timo Järvinen (701/8000, 15.8× lift)`.

Why this matters: high-precision patterns can be promoted to deterministic rules. The hybrid path — mined rules first, `_predict` fallback — gives the AP team auditable routing on the predictable 70% and Aito's predictions on the long tail. Both share the same data, no duplication.

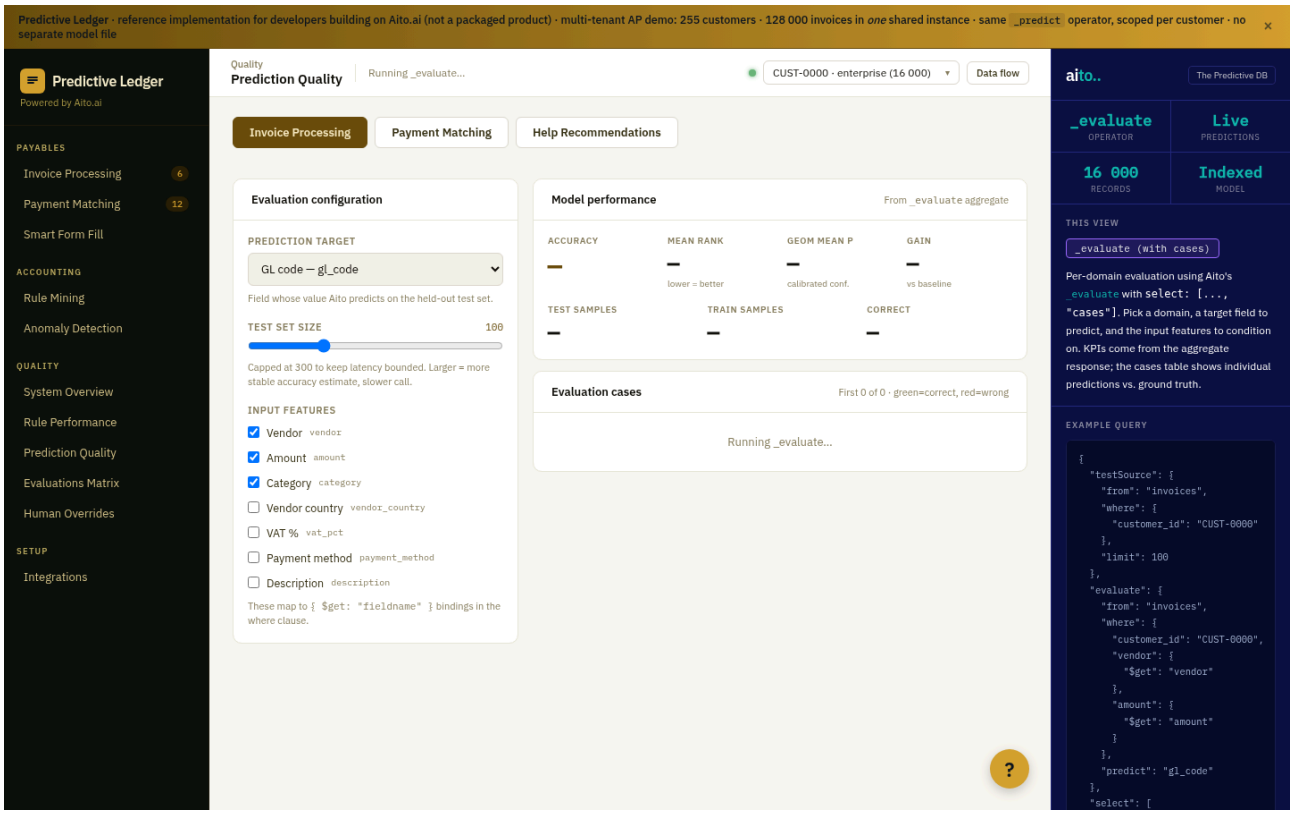
Anomaly detection



Inverse prediction. For each posted invoice, `_predict` against the ground-truth GL — if the actual GL has low probability under the model, flag it. Same operator as Invoice Processing, used to score historical posts instead of route new ones.

What it catches: miscoded invoices, vendor + category mismatches, amount outliers. No separate anomaly model — the same probability that drives routing also detects deviations.

Quality dashboard

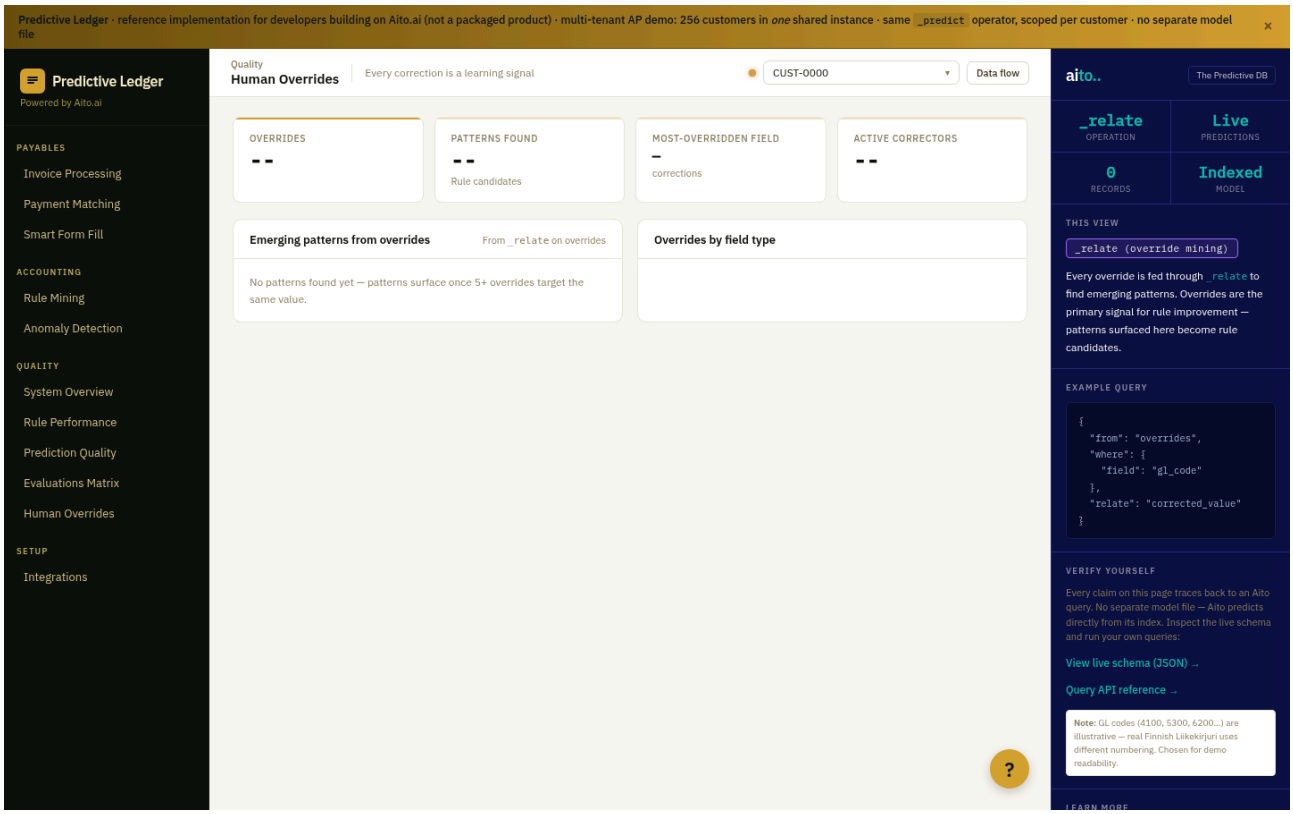


Pick a domain (Invoice / Payment / Help), a target field, the input features. The page runs `_evaluate` with `select: [..., "cases"]` against a held-out test sample and renders a green/red diff table per case. **Including the failures.**

> **GL code accuracy** (CUST-0000, 50-sample test): **98%** (+44pp over baseline)
> **Approver accuracy** (same sample): **80%** (+44pp over baseline)
> **Geom mean p**: 0.76 — well-calibrated; top hits really are 76% confident on average

The headline number is the aggregate; the cases table is the auditable evidence. Procurement reviews ask for both.

Human overrides → rule candidates



When a controller corrects a predicted GL, the override lands in `overrides` table. Two-pass `_relate` mines those corrections for input-driven patterns: pass one finds the most-corrected target values, pass two walks the schema link (`overrides.invoice_id.vendor`) to surface the input that drove the correction.

Output rows have the same `input → output` shape as Rule Mining, so they can be promoted directly into the rules table. **The system gets sharper as people use it. Zero retraining step.**

How it works

📥 1. Connect your data

GL export from your ERP → Aito instance. Standard CSV format. Schema auto-detected. The demo's schema (invoices, bank_transactions, overrides, customers, employees) is a starting point.

🔮 2. Use the operators

_predict for routing, _relate for rule mining, _recommend for in-app help, _evaluate for accuracy measurement. All take where: {customer_id} for tenant scoping.

🚢 3. Integrate or ship

REST API, no SDK required. Reference frontend (Next.js) and backend (FastAPI) in this repo. Deploy via Docker — see docs/deploy-azure.md.

Performance & sizing

Operator latencies (server work, warm conn):

- `_search` 20 hits: **22 ms**
- `_predict gl_code`: **57 ms**
- `_relate` 5 hits: **17 ms**
- `_evaluate` 50 samples: **8 s**

The headline: indexed reads are millisecond-class. `_evaluate` is the only slow operator — and the demo precomputes it.

Sizing rules of thumb:

- 128 K invoices → 150 MB RAM, 400 MB disk
- 1 M invoices → 1 GB RAM, 3 GB disk
- 10 M invoices → 8 GB RAM, 25 GB disk
- 100 M invoices → 60 GB RAM, 200 GB disk

Memory grows linearly with row × indexed-field count. See `docs/sizing.md` for the full table.

Open source · auditable · fixed licence

Predictive Ledger is published as a public reference implementation.

Source: github.com/AitoDotAI/aito-accounting-demo
Companion: github.com/AitoDotAI/aito-demo (e-commerce)

Every claim in this document is traceable to running code, an ADR in `docs/adr/`, or a measured result in `book/test_*.py`. The demo runs on a free Aito instance — `./do dev` after a 5-step quickstart.

Aito ships as a fixed-licence, on-premise product — no per-query metering, no data residency surprises, and an explicit migration playbook (`docs/migration.md`) if you ever need to leave.

hello@aito.ai · aito.ai

Aito.ai is a Helsinki-based predictive database company.
Reference implementation maintained by the Aito team.